Architectural Decisions

- Definition
  - Mozaic, the system, and Mozaic system refers to the software system that this project is going to develop, launch, and operate.
- Goal
  - This document describes architectural decisions that implement the system requirements.
  - This document serves as additional technical terminology of the project.

1. Overall state machine

1.1 Definition

- **Asset state** is the state of what amounts of what tokens, including LP tokens, on what chains are, at the given moment of time,
  - deposited and pending staking
  - staked
  - rewarded and pending harvesting
  - staying in system treasuries
  - pending withdrawal
- **Request state** is the state of what deposit/withdrawal requests are accepted by the system and waiting for final processing, at the given moment of time.
- If the system does **Optimize asset/request state**, it changes the states to earn optimal staking reward.

1.2 Design decisions

We choose the Sleeping-then_Optimizing model for the overall system behavior.

- The system will not always be optimizing, but sleeping most of the time accepting deposit/withdrawal requests from users.
- If the system **accept**s a deposit request, it
  - collects the asset the user wants to deposit, (on the asset's chain)
  - tells the user to wait until the next optimization round, when the system will send LP tokens to the user in return for the asset,
  - and book the request with the system for later processing.
- If the system **accept**s a withdraw request, it
  - collects the LP tokens the user wants to return, (on the LP's chain)
  - tells the user to wait until the next optimization round, when the system will send some asset of the requested token type,
  - and book the request with the system for later processing.
- The system will **optimize asset/request state** at regular or irregular intervals. The frequency of optimization rounds will be optimized, as frequent moves of asset may incur more costs while infrequent optimization rounds will hinder from quick

maneuver of staking.

- When entering the **Optimization process**, or the **Optimizing** state, the system will take a snapshot of the asset/request state. Then the system transforms/changes the states to optimal states for maximum reward, while continuing to **accept** deposit/withdrawal requests, which will be handled at the next round of optimization.

1.3 Visual description

The Sleeping-then_Optimizing model of behavior can be expressed in a UML State Machine diagram shown below:

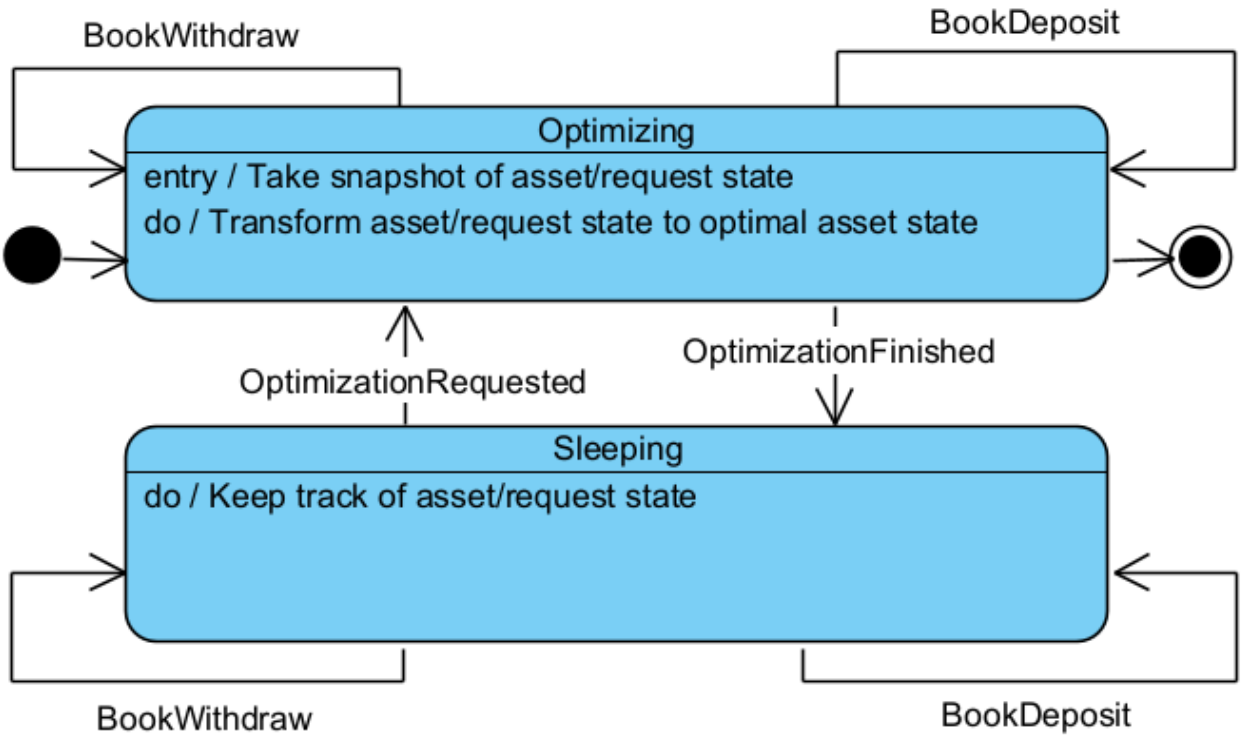


Figure. State Machine of Mozaic asset management

2. Vaults' local responsibility

We need a module that implements the use case **Control asset move** identified in the requirements specification, solely, and completely. We call the module the vault, for the following reasons.

2.1 Vaults are smart contracts

According to the requirements, vaults are responsible to, exclusively and at the decentralization level,

- keep track of all changes to asset/request state, defined in the previous section
- execute all changes to asset/request state,
- log all changes to asset/request state.

The only way is to deploy smart contracts on chains that cooperate with each other to form an abstract vault module. We call them vaults or vault contracts.

2.2 Vaults have limited responsibility

According to the requirements, vaults do *not* have to

- find an optimal asset state to **Optimize asset/request state** to
- execute asset move requests, like assetMovePlan identified in requirements, strictly, because there will not be negative profit.

2.3 Vaults are responsible for local transportation of tokens to/from wallets

- Pull asset from the user if a deposit involves a local token of asset to collect
- Push LP to the user if a deposit involves the local LP token to return
- Pull LP tokens from the user if a withdrawal involves the local LP token to collect
- Push asset to the user if a withdrawal involves a local token of asset to return
- Swap at local Dex pools

2.4 Vaults are responsible for local moves of Staking stock

- Stake assets to local staking pools
- Un-stake whole or partial assets from local staking pools
- Collect rewards from local staking pools
- Swap at local Dex pools

3. Architecture for omnichain staking

3.1 Definition**

Omnichain staking requires that: - Assets can be deposited in any listed token format on any listed chain, *at users' request*. - Deposited assets can be swapped/transferred, and staked in any staking pool on any listed chain, *guided by the system's optimization plan.* - Staked assets and rewards can be withdrawn in any listed token format on any listed chian, *at users' request.* - Rewards collected can be swapped/transferred, and staked in any staking pool on any listed chain, *guided by the system's optimization plan.*

For a given chain, we define the followings:

$ChainAssetPlaces = \{ChainVaultWallet\} \cup ChainStakingPools \cup ChainWithdrawalWallets$

- $ChainVaultWallet$: the vault's wallet on the given chain. They have
  - deposited assets that are pending staking
- $ChainStakingPools$: all staking pools on the given chain. They have
  - staked assets
  - pending rewards
- $ChainWithdrawalWallets$: wallets of all users whose withdrawal requests are booked and who will receive withdrawal assets on the given chain. They have
  - negative undefined asset to send to the users - the system has to calculate the amount based on the LP token amount redeemed from the user.

$AllVaultWallets$: the set of $ChainVaultWallet$ of all listed chains.

$AllStakingPools$: the sum of $ChainStakingPools$ of all listed chains.

$AllWithdrawalWallets$: the sum of $ChainWithdrawalWallets$ of all listed chains.

$AllAssetPlaces = AllVaultWallets \cup AllStakingPools \cup AllWithdrawalWallets$

We know an asset move is the move of asset, whether it changes the token type or not, between any two of $AllAssetPlaces$.

We classify asset moves into the following two groups:

- **Inter-Chain Moves** of a given chain: asset moves that take place between any two of $ChianAssetPlace$ of the given chain
- **Intra-Chain Moves**: asset moves that take place between one of a chain's $ChainAssetPlaces$ and another of another chain's $ChainAssetPlaces$

3.2 Irregular and regular asset moves

From the perspective of the cost of asset moves, we have irregular asset move plan and regular asset move plan.

Example of irregular asset move plan

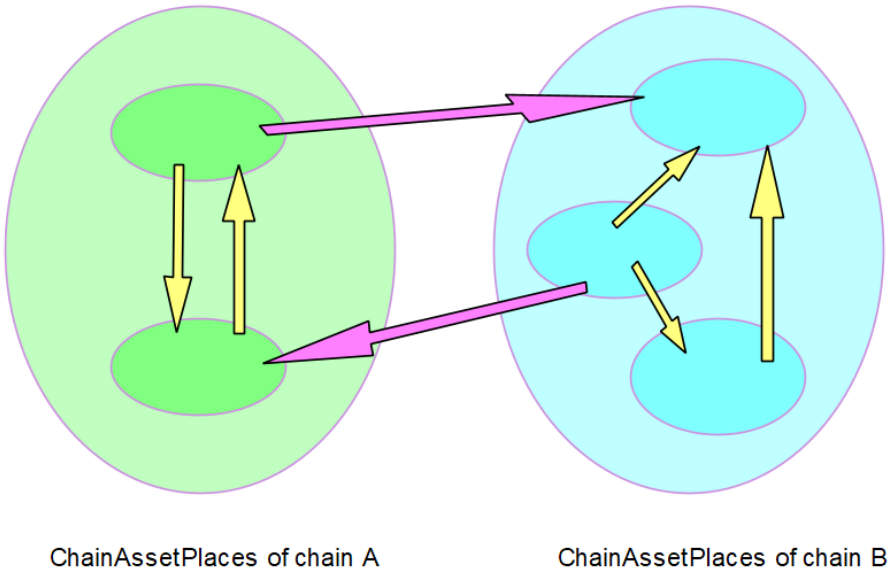


Figure. Irregular asset moves. (swaps are hidden)
Intra-Chain Moves and Inter-Chain Moves have cyclical value moves.
Some asset places both give and take, for example.

Example of regular asset move plan

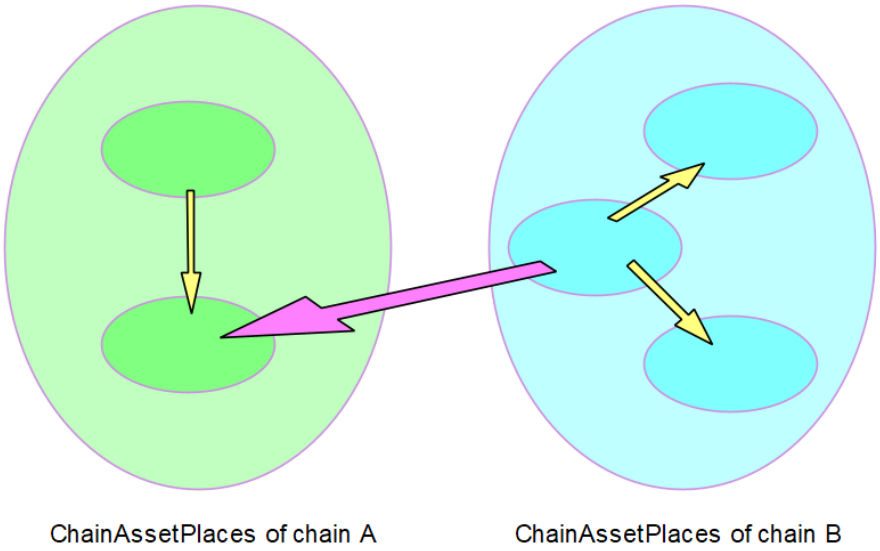


Figure. Regular asset moves. (swaps are hidden)
Intra-Chain Moves and Inter-Chain Moves have no cyclical value moves.
An asset place can ether only give or only take.

3.3 Design decisions

We deduce the following design decisions:

- Asst moves will be regularized. We believe regularization will reduce the total cost of asset moves and the number of inter-chain swap/transfers.
- We believe that ChainWallet will be the best place for regular **Inter-Chain Moves**. It means **Inter-Chain Moves** will be between ChainWallets.
- We need one special vault that oversees the cooperation between vaults, including itself, at decentralization level.
    - **Master vault**: the special vault
    - **Home chain**: the chain that hosts the master vault

3.4 Directional algorithm for upgrading staking

This algorithm will be implemented off-chain, because

- it will save gas fees
- the requirements don't require decentralization-level of asset move planning

This algorithm handles assets:

- in their USDC-equivalent on the chain, rather than in their own token, when working intra-chain
- in their equivalent of home chains' USDC or the largest giving chains's USDC
- we hope USDC on all chains will have exactly the same price

Algorithm

- Input: Target staking portfolio. (See the coming sections for optimal staking portfolio)
- Classify asset places into giving places and taking places
    - If the current asset amount is significantly greater than the target asset amount, it is a giving asset place
    - If the current asset amount is significantly less than the target asset amount, it is a giving asset place
    - Else, it is a neutral asset place
    - An asset place has
        - a positive giving amount if it is a giving asset place, else zero
        - a positive taking amount if it is a taking asset place, else zero
        - a zero giving amount and a zero taking amount if it is a neutral asset place.
- Classify chains into giving chains and taking chains
    - If the sum of giving amount of all ChainAsstPlaces is significantly greater than the sum of taking amount, it is a giving chain
    - If the sum of giving amount of all ChainAsstPlaces is significantly less than the sum of taking amount, it is a giving chain
    - Else, it is a neutral chian
    - A chain has

        - a positive giving amount if it is a giving chain, else zero
        - a positive giving amount if it is a taking chain, else zero
        - a zero giving amount and a zero taking amount if it is a neutral chain
- Generate a regular inter-chain asset move plan, AesseMovePlan, for giving chains
    - Collect the local swap prices and fees
    - Sum up surplus asset, which is the target asset less the current asset, of giving asset places to a variable
    - Make a distribution plan that divides the sum of surplus asset to taking asset places
    - Do not care of deficit amount. (Leave it to vaults' tunning operation)
    - Now, an irregular asset move plan has been created
    - Regularize the plan, by using graph theories and ad-hoc techniques
    - Conclude with the surplus assets amount of this giving chain
    - **Be aware that the asset move plan is actually asset send plan that doesn't care of price slippage and fees**
    - **Be aware that this plan does/can not take into account the actual price slippage and fees**
- Transfer surplus assets on giving chains to taking chains
    - Sum up surplus assets of giving chains to a variable
    - Make a distribution plan that divides the sum of surplus assets to taking chains
    - Do not care of deficit amount. (Leave it to vaults' tunning operation)
    - Now, an irregular asset move plan has been created
    - Regularize the plan, by using graph theories and ad-hoc techniques
    - **Be aware that the asset move plan is actually asset send plan that doesn't care of price slippage and fees**
    - **Be aware that this plan does/can not take into account the actual price slippage and fees**
- Generate a regular inter-chain asset move plan, AeetMovePlan, for taking chains
  (the same as for giving chains)

4. Architecture for omnichain LP token

4.1 Definition**

Omnichain LP requires that:

- The LP token should exist on all listed chains.
- All LP token versions should always have the save value.

4. Logical components layout

The design decisions are as illustrated in the following figure:

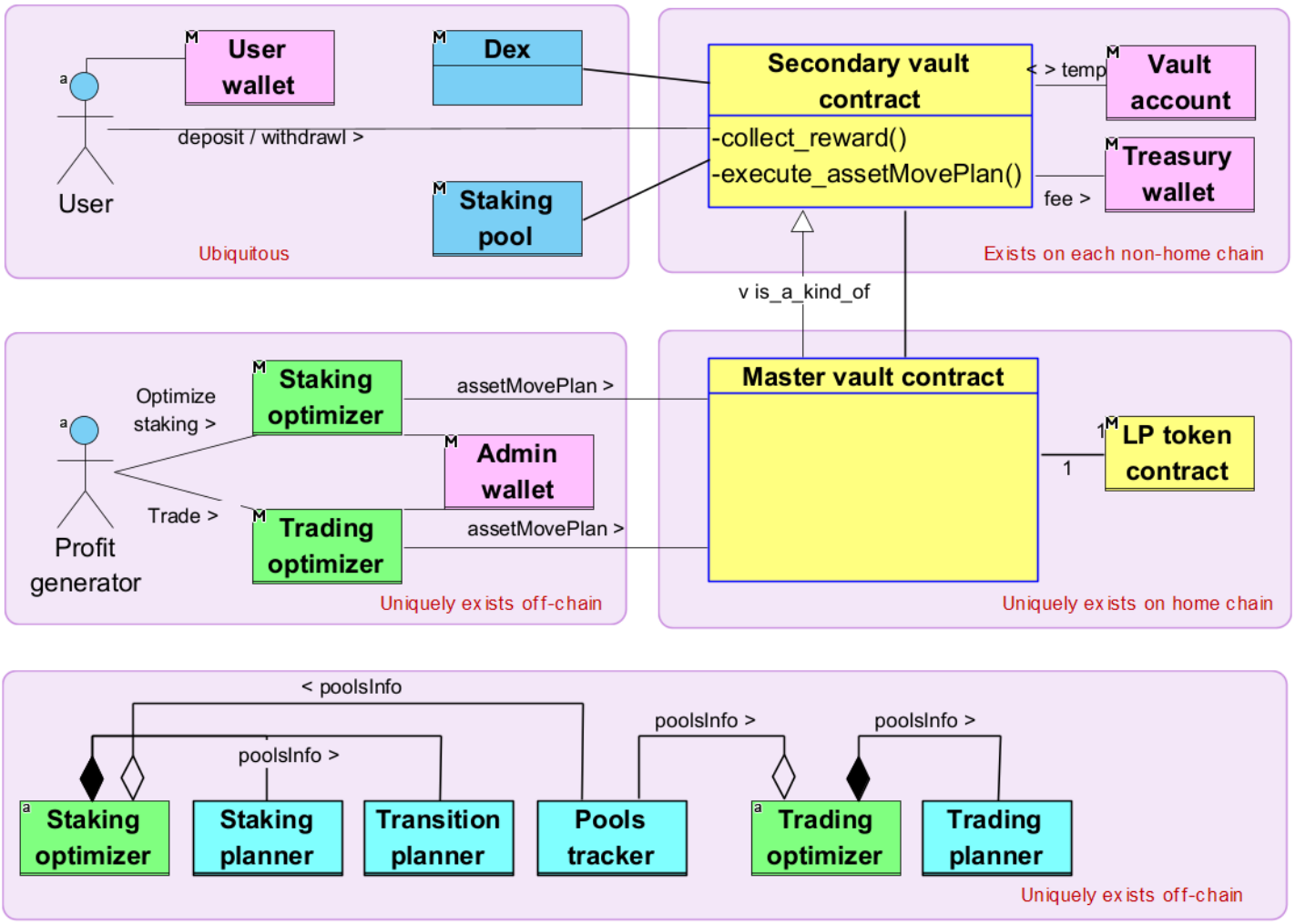


Figure. High-level functional modules of Mozaic DeFi ver. 1.0
Goal: Minimize the functionality of vaults.

Functional modules are described below:

- **Secondary vault contract**: This module is a smart contract and deployed on each chain except the home chain. It participates in the cooperation with its peer **Secondary vault contract**s to implement **Control asset move**, This module has at least the following private operations that work locally on its chain: collect_reward(...) and move_staking_asset(...)
- **Master vault contract**: This module is a smart contract, has global operations, in addition to the local operations inherited from **Secondary vault contract**, and is deployed on one and only one of the chains, called **Home chain**, where it takes the role of **Secondary vault contract**, as well as the the unique role of the master vault operating **LP token contract**.
- **LP token contract**: This module manages the LP token balances of **User**s, which represents their proportional share of the total assets of the system.

- **User wallet**: This is a blockchain wallet and identifies a **User**. **User**'s actions; like deposit, withdraw, and harvest; are authenticated/authorized with this wallet.
- **Vault account**: It is the blockchain account of, and controlled by, **Secondary vault contract** and used to store temporary assets, like funds pending staking.
- **Treasury wallet**: This is a blockchain wallet, and a place to store and retrieve system revenues, like fees. It will be better if it is not owned by a human, but be the account of a smart contract that only obeys vault contracts, for better decentralization. It is deployed on all chains.
- **Staking optimizer**: This is an off-chain module that can invoke **Master vault contract**s. This module is globally unique, calculates optimal **assetMovePlan**s, and lets the master vault to execute the plans (in cooperation with secondary vaults). *It is an important design decision that the assetMovePlan is calculated off-chain, thus leading to transparency and security debates, for the sake of gas- and time- savings:*
    - Transparency debate: **User**s will not be able to track why the system chose particular **assetMovePlan**s technically.
    - Security debate: If the calculation of **assetMovePlan** is hacked or compromised, then the system will make a less-optimal staking maneuver.
    - Justification: Only the second of the following concerns becomes less transparent, leading to both un-assured best profitability and assured huge gas- and time- savings.
        - how much of what assets from which pool to which pool, is the move about
        - whether all the asset moves are securely and/or reasonably/optimally chosen
        - whether all the asset moves are securely executed and logged
        - whether the move logs are readily available to check later
        - whether the execution of assetMovePlan is integrated
- **Trading optimizer**: This off-chain module is similar to **Staking optimizer**, except it relates to trading.
- **Adimin wallet**: This wallet is used to invoke **Master vault contract", in privilege, on behalf of the administrator.
- **Staking planner**: An integral component of **Staking optimizer**, this module predicts the next most profitable **staking_portfolio**, based on **poolsInfo** provided by **Pools tracker**. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fees and effectively disable the system.
- **Transition planner**: An integral component of **Staking optimizer**, this module predicts the most efficient **assetMovePlan**, which is the best procedure of asset move that implements the transitioning to a given **staking_portfolio**, based on the current **poolsInfo**.
- **Trading optimizer**: This is similar to **Staking optimizer**, except that it relates to trading.
- **Trading planner**: This is similar to **Staking planner**, except that it relates to trading.
- **Pools tracker**: A shared module between **Staking optimizer** and **Trading optimizer**, this module retrieves and tracks all relevant information from chains, like Reward Release Speed, and total Staked LP of each pool. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fees and effectively disable the system.

5. Exploring vaults

We have identified vaults through their surrounding modules interacting with them.
The external actors in the following use case diagram, together with their interactions with vaults are already described above.
We can now explore the use cases of vault.

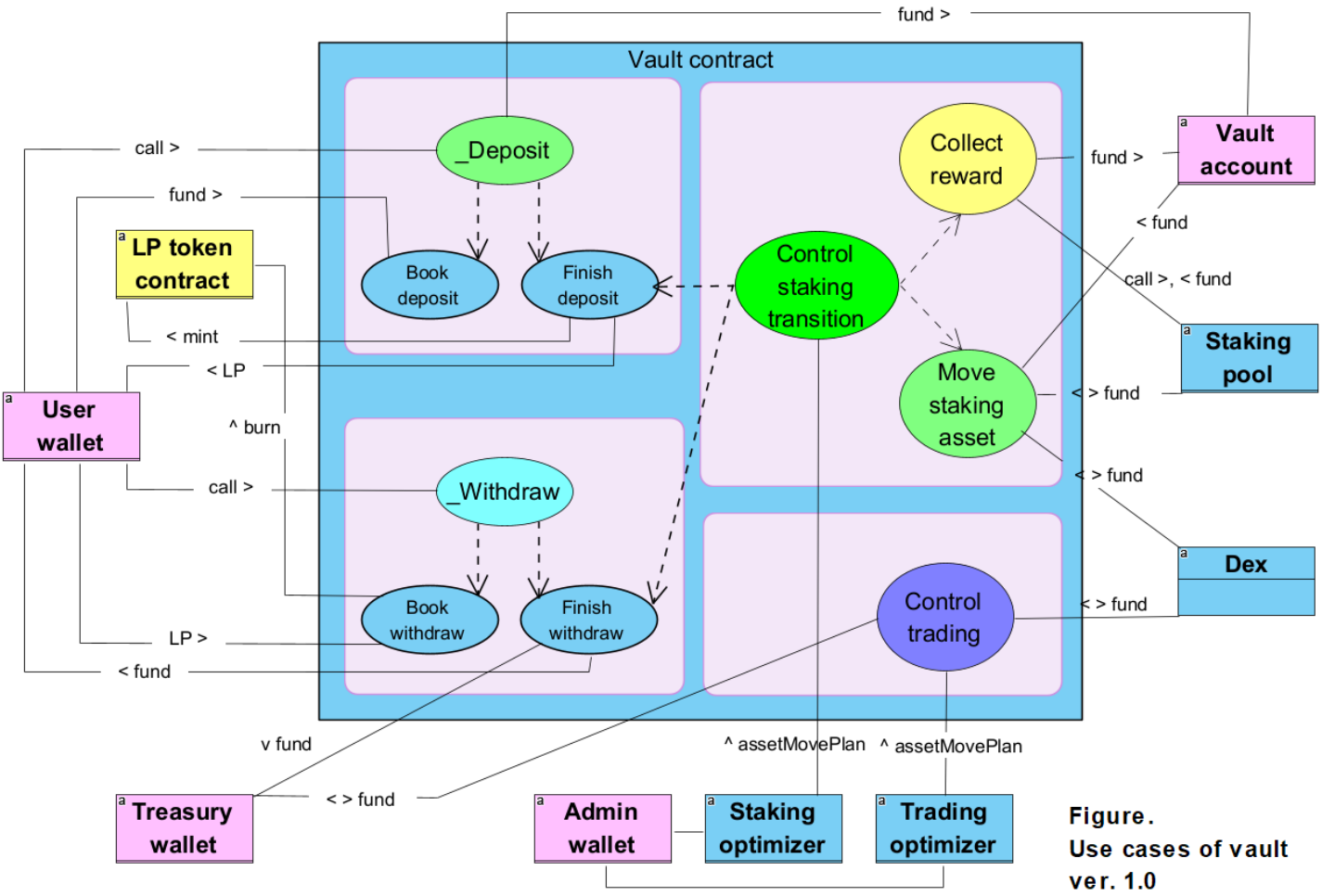


Figure.
Use cases of vault ver. 1.0

- **_Deposit**: This is what happens at the level of vault contracts when the **Deposit** use case is invoked at the system level. Invoked by the **User wallet** with fund amount to deposit, this use case coordinates the following two use cases.
- **Book deposit**: This use case
  - collects the fund from **User wallet**,
  - books the deposit request with the system,
  - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
  - resume the session of "_Deposit",
  - retrieves the booked deposit request,
  - mints LP tokens to cover the new fund,
  - returns the LP tokens to **User wallet**,
  - and helps **Control staking transition** stake the fund.
- **_Withdraw**: This is what happens at the level of vault contracts when the **Withdraw** use case is invoked at the system level. Invoked by **User wallet** with LP tokens returned, this used case coordinates the following two use cases.
- **Book withdraw**: This use case

    - collects the returned LP tokens,
    - burns the collected LP tokens,
    - books the withdrawal request with the system,
    - and pauses the session of "_Deposit".
- **Finish deposit**: This use case
    - resume the session of "_Deposit",
    - retrieves the books withdrawal request,
    - subtract fund, as much as covered by the returned LP tokens, from the total system assets, and
    - returns the fund to **User wallet**
- **Control staking transition**: This use case transitions to a new asset state by executing **assetMovePlan** provided by **Staking optimizer**. (This is the most challenging part of vault implementation.) It does *collectively*, by using **Move staking asset**,
    - **Collect reward**,
    - Collect staked assets, to cover the fund to **Finish withdraw**,
    - **Finish deposit**,
    - **Finish withdraw**,
    - execute remaining part of **assetMovePlan**
- **Control trading**: This use case executes **assetMovePlan** provided by **Trading optimizer**.

6. Algorithm of Staking planner, for optimal staking portfolio

Note. All errors, like numerical processing rounding and price slippage, are ignored at this stage of architectural design.

6.1 Task definition

- Goal: Calculate best staking portfolio, in order to
  - Save vault contracts long calculations of staking optimization, thus to save gas.
  - Keep vault contracts insulated from future algorithm upgrades of staking optimization.
- Consideration
  - Input may not be idealistically consistent inside itself, because an idealistic snapshot of multiple chain states is impossible logically.
  - Output staking portfolio may not be completely/perfectly implemented, because input may have errors and there are unpredictable price slippages sneaked into the calculation.

- Input
  - Deposit requests currently booked
  - Withdrawal requests currently booked
  - Current pending rewards
  - Current poolsInfo

- Process
  (See below)

- Output
  - optimal staking portfolio

6.2 Definition

- **Pools state**
  $PS = \{PS_i | i = 1..N\}$,
  where PS_i is the i-th pool state:
  $PS_i = (RewardRate_i, RewardToken_i, TotalStake_i, StakingToken_i, MozaicStake_i)$
  Note: We assume reward on PS_i is calculated as:
  $MozaicReward_i = \frac{RewardRate_i}{TotakStake_i} \times MozaicStake_i$

- **Token vectors**
  - **User tokens vector**
    $UserTokens = \{UserToken_i | i = 1..M\}$
    Example: $UserTokens = \{USDT, USDC, ETH, BNB, BTC\}$
  - **Reward tokens vector**
    $RewardTokens = \{RewardToken_i | i = 1..N\}$
  - **Staking tokens vector**
    $StakingTokens = \{StakingToken_i | i = 1..N\}$

- **Tokens vector**
  $Tokens = (UserTokens, RewardTokens, StakingTokens)$

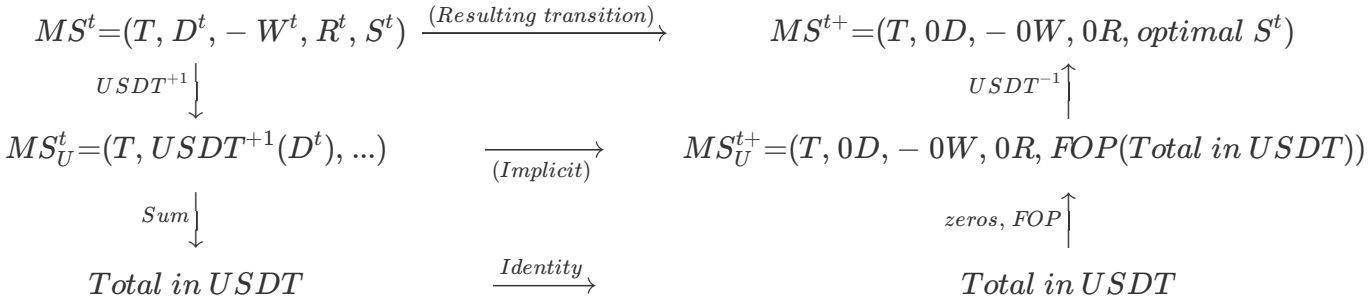
- **Asset vectors**
  - **Vector of booked deposit requests**, at time index t
    $Deposits^t = \{D_i^t | D_i^t : deposit\ amount, at\ time\ t,\ denominated\ by\ UserToken_i.i = 1..M\}$
    Example: $Deposits^t$ of {1, 2, 3} means {1 USDT, 2 USDC, 3 ETH}, assuming $UserTokens = \{USDT, USDC, ETH\}$
  - **Vector of booked withdrawals requests**, at time index t
    $Withdrawals^t = \{W_i^t | D_i^t : withdrawal\ amount, at\ time\ t,\ denominated\ by\ UserToken_i.i = 1..M\}$
  - **Vector of collected rewards**, at time index t
    $Rewards^t = \{R_i^t | D_i^t : reward\ amount, at\ time\ t,\ denominated\ by\ RewardToken_i.i = 1..M\}$
  - **Vector of staked assets**, at time index t
    $Stakes^t = \{S_i^t | D_i^t : stake\ amount, at\ time\ t,\ denominated\ by\ StakingToken_i.i = 1..M\}$

- **Transformations**
  - **Transformation $USDT^{+1}$**
    $USDT^{+1}$ transforms a Tokens-denominated asset vector to USDT-denominated vector, with market exchange rates.
    Example: $USDT^{+}$ transforms (1, 2, 3) to (1, 2.02, 1300), assuming UserTokens = (USDT, USDC, ETH) and USDT/USDC = 1.01 and USDT/ETH = 1300.
  - **Transformation $USDT^{-1}$**
    $USDT^{-1}$ transforms a USDT-denominated asset vector to Tokens-denominated vector, with market exchange rates.
    Example: $USDT^{-1}$ transforms (1, 2.02, 1300) to (1, 2, 3), assuming UserTokens = (USDT, USDC, ETH) and USDT/USDC = 1.01 and USDT/ETH = 1300.
  - **Transformation FOP**
    FOP, standing for Find Optimal Portfolio, finds the best USDT-denominated vector of staked assets, for a given total USDT amount.
    Example: FOP transforms (12345) to the best staking of 123 USDT over staking pools, and has the format of USDT-denominated $Stakes^t$, like (100, 20, 3) all in USDT, assuming we have a total of 3 pools.
  - **Transformation Sum**
    $ElementWiseSum$ sums up all elements of a vector.

- **Mozaic's asset state**, at transition t
  - Asset snapshot just before the transition that takes place at time t:
    $MS^t = (Tokens, Deposits^t, -Withdrawals^t, Rewards^t, Stakes^t)$
    Or, simply, $MS^t = (T, D^t, -W^t, R^t, S^t)$
  - Asset snapshot just after the transition that takes place at time t:
    $MS^{t+} = (Tokens, 0Deposits, -0Withdrawals, 0Rewards, optimal\ Stakes^t)$
    Or, simply, $MS^{t+} = (T, 0D, -0W, 0R, optimal\ S^t)$
    , where 0D, 0D, and 0R are a vector of zero values in their respective vector lengths.

6.3 Formulae

If a state transition takes place at time t, Mozaic's asset state MS^t changes to MS^{t+} as shown in the following diagram:



The algorithm for Staking planner MS^t is a chain of transformations:

$$optimial\ S^t = USDT^{-1} \circ FOP \circ Sum \circ USDT^{+1}(T,\ D^t,\ -W^t,\ R^t,\ S^t)$$

- $USDT^{+1}$ and $USDT^{-1}$ are obvious, except that we may need systematic methods to find best Dexes and swap paths.
- FOP is solved analytically, demonstrating about 9% of competitive edge over the public.
- ElementWiseSum is trivial.
- $D^t and W^t$ can be retrieved from the booked requests of deposits and withdrawals.
- S^t is found when we "Collect reward" pending rewards.

The formula essentially does:

- collect all available assets: deposits pending staking, less withdrawals pending, plus rending rewards, and the current staking.
- transform them to USDT and sum up to get a single number: Total in USDT.
- allocate the Total in USDT optimally across all staking pools.
- transform the allocated USDT amount back to the native tokens for the staking pools.